COMPSCI 572.01 (NLP) Pretraining 3

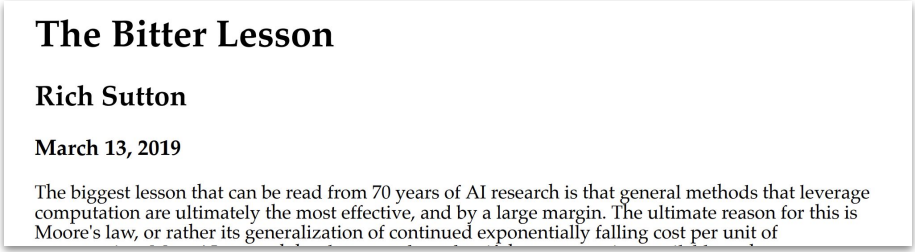
Scaling Laws

Outline

- Scaling laws
  - Basics
  - Neural scaling laws (Kaplan et al, 2020)
  - Chinchilla scaling laws (Hoffman et al, 2022)
- Scaling model size
  - Dense models: different types of parallelism
  - Sparse models: mixture-of-experts

Why scaling?

- History of language modeling is an effort on **scaling**
  GPT-1 (117M) → BERT (340M) → GPT-2 (1.5B) → T5 (11B) → GPT-3 (175B)



The Bitter Lesson

Rich Sutton

March 13, 2019

The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is Moore's law, or rather its generalization of continued exponentially falling cost per unit of

Why scaling?

- History of language modeling is an effort on **scaling**
  GPT-1 (117M) → BERT (340M) → GPT-2 (1.5B) → T5 (11B) → GPT-3 (175B)

- Training large models is expensive
  - GPT-3 estimated cost to train (final run): $5-10M
  - You don't get many attempts to do this

- Training large models is difficult
  - Should we train wide or deep transformers?
  - How much regularization? Which position embeddings?
  - How much gain in accuracy can I expect from my $$$ investment?

Why scaling?

- History of language modeling is an effort on **scaling**
  GPT-1 (117M) → BERT (340M) → GPT-2 (1.5B) → T5 (11B) → GPT-3 (175B)

- Training large models is expensive
  - GPT-3 estimated cost to train (final run): $5-10M
  - You don't get many attempts to do this

- Training large models is difficult
  - Should we train wide or deep transformers?
  - How much regularization? Which position embeddings?
  - How much gain in accuracy can I expect from my $$$ investment?

Scaling laws attempt to **predict large model performance by extrapolating** from smaller models

Compute: the main constraint

- Large model training is constrained by:
  - How many GPUs are available
  - For how long

- Together these define how many FLOPS you can spend
  - FLOPS = Floating point operations per second
  - In practice the theoretical peak is hard to achieve due to communication overhead
  - This is where a lot of systems engineering comes in

Technical Specifications	
	H100 SXM
FP64	34 teraFLOPS
FP64 Tensor Core	67 teraFLOPS
FP32	67 teraFLOPS
TF32 Tensor Core*	989 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS

Rule of thumb for compute

For Transformer models: Training FLOPS = 6 x N x T

- N = Number of parameters
- T = Total tokens seen during training (including repetitions)

(number of steps x batch size in tokens)

Rule of thumb for compute

For Transformer models: Training FLOPS = 6 x N x T

- N = Number of parameters
- T = Total tokens seen during training (including repetitions)

E.g., T5-11B → 6 x 11B x 1T ~= 6 x 10^22 FLOPs
⇒ 4 x 10^6 seconds (46 days) on 8xH100 GPUs (assuming theoretical peak)

Technical Specifications	
	H100 SXM
FP64	34 teraFLOPS
FP64 Tensor Core	67 teraFLOPS
FP32	67 teraFLOPS
TF32 Tensor Core*	989 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS

Neural Scaling Laws (Kaplan et al, 2020)

Goal: Give a fixed amount of compute, how should we allocate it?

Idea: let's train a series of smaller LMs and see if there is a relationship between hyperparameters and loss

Neural Scaling Laws (Kaplan et al, 2020)

Goal: Give a fixed amount of compute, how should we allocate it?

Idea: let's train a series of smaller LMs and see if there is a relationship between hyperparameters and loss

Procedure:

To characterize language model scaling we train a wide variety of models, varying a number of factors including:

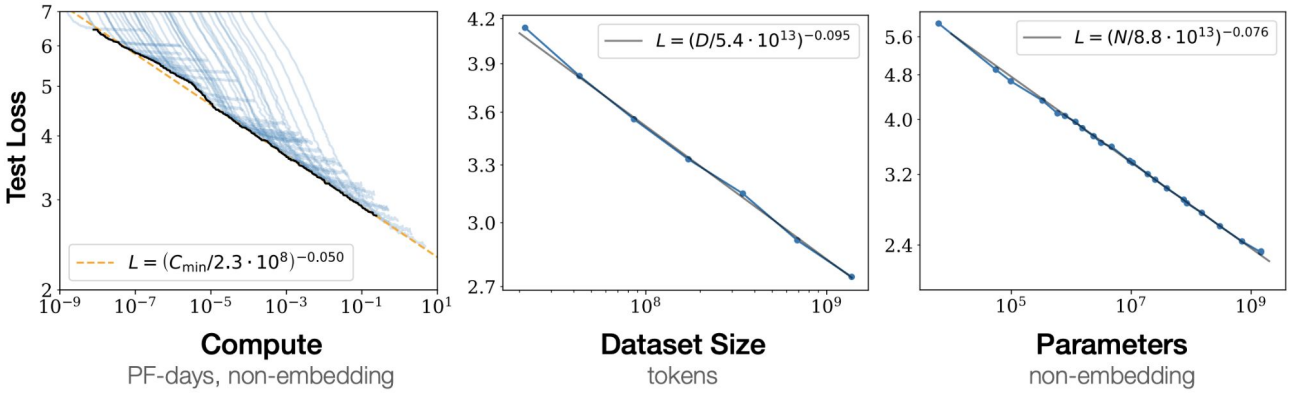
- Model size (ranging in size from 768 to 1.5 billion non-embedding parameters)
- Dataset size (ranging from 22 million to 23 billion tokens)
- Shape (including depth, width, attention heads, and feed-forward dimension)
- Context length (1024 for most runs, though we also experiment with shorter contexts)
- Batch size (2^{19} for most runs, but we also vary it to measure the critical batch size)

Neural Scaling Laws (Kaplan et al, 2020)

Goal: Give a fixed amount of compute, how should we allocate it?

Idea: let's train a series of smaller LMs and see if there is a relationship between hyperparameters and loss

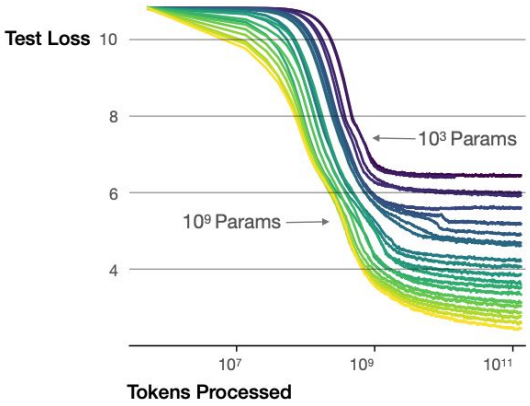
Result: Power-law relationship between loss vs optimal compute / data size / model size



Neural Scaling Laws (Kaplan et al, 2020)

Observations:

1. Larger models are more sample-efficient



Neural Scaling Laws (Kaplan et al, 2020)

Observations:

1. Larger models are more sample-efficient
2. How the model size is scaled doesn't matter much

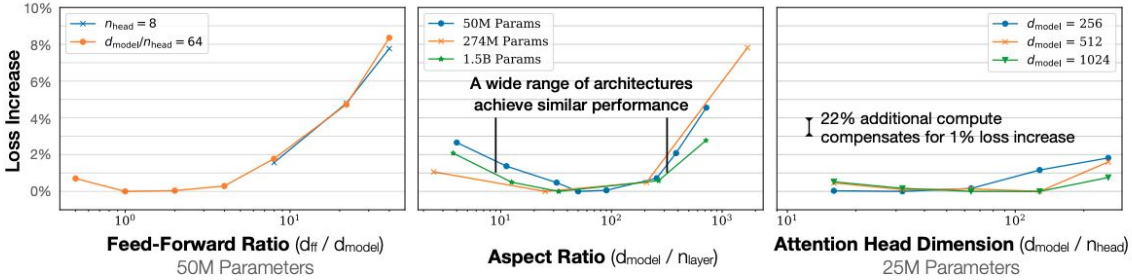
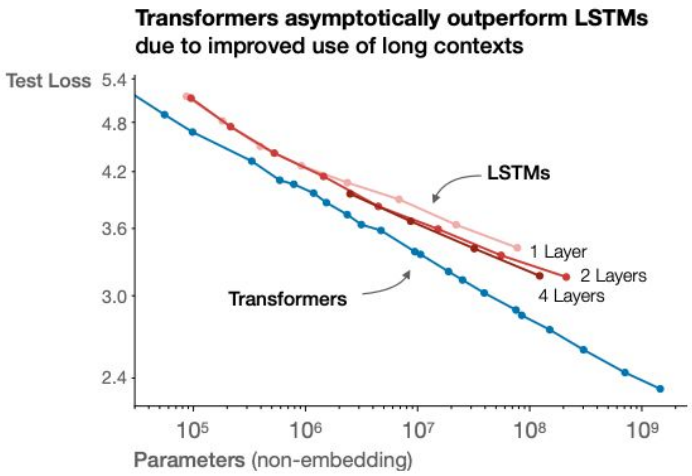


Figure 5 Performance depends very mildly on model shape when the total number of non-embedding parameters N is held fixed. The loss varies only a few percent over a wide range of shapes. Small differences in parameter counts are compensated for by using the fit to $L(N)$ as a baseline. Aspect ratio in particular can vary by a factor of 40 while only slightly impacting performance; an $(n_{\text{layer}}, d_{\text{model}}) = (6, 4288)$ reaches a loss within 3% of the $(48, 1600)$ model used in [RWC+19].

Neural Scaling Laws (Kaplan et al, 2020)

Observations:

1. Larger models are more sample-efficient
2. How the model size is scaled doesn't matter much
3. Transformers have better scaling behavior than LSTMs

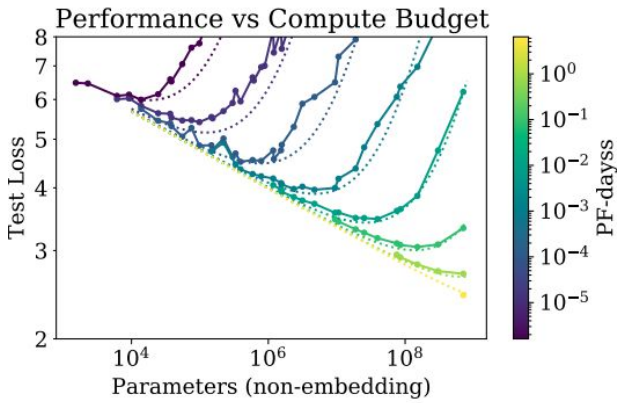


Neural Scaling Laws (Kaplan et al, 2020)

Observations:

1. Larger models are more sample-efficient
2. How the model size is scaled doesn't matter much
3. Transformers have better scaling behavior than LSTMs
4. Given a compute budget we can predict the optimal model size

Note: these experiments assume **no data repetition**



But scaling laws can be wrong

- Kaplan et al concluded that as compute increases, model size should be scaled faster than data size

$$N_{opt}(C), D_{opt}(C) = \underset{N,D \text{ s.t. } \mathrm{FLOPs}(N,D)=C}{\operatorname{argmin}} L(N, D).$$

Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

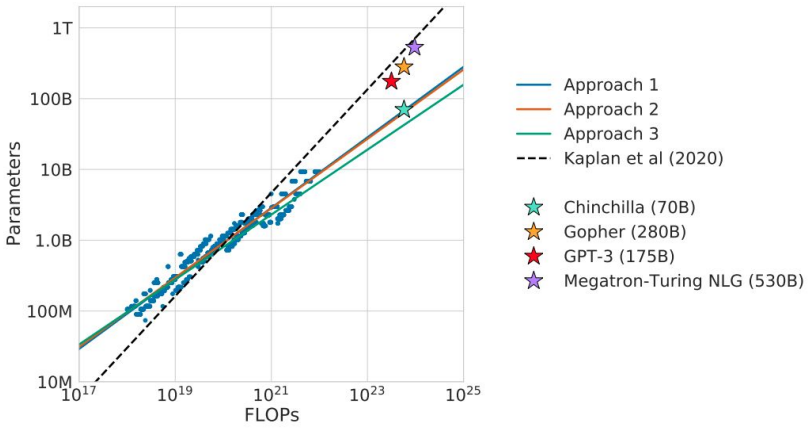
But scaling laws can be wrong

- Kaplan et al concluded that as compute increases, model size should be scaled faster than data size

- Chinchilla (Hoffman et al, 2022) challenges this:
  - They show that as compute increases, **model and data size should scale equally**

$$N_{opt}(C), D_{opt}(C) = \underset{N,D \text{ s.t. } \mathrm{FLOPs}(N,D)=C}{\operatorname{argmin}} L(N, D).$$

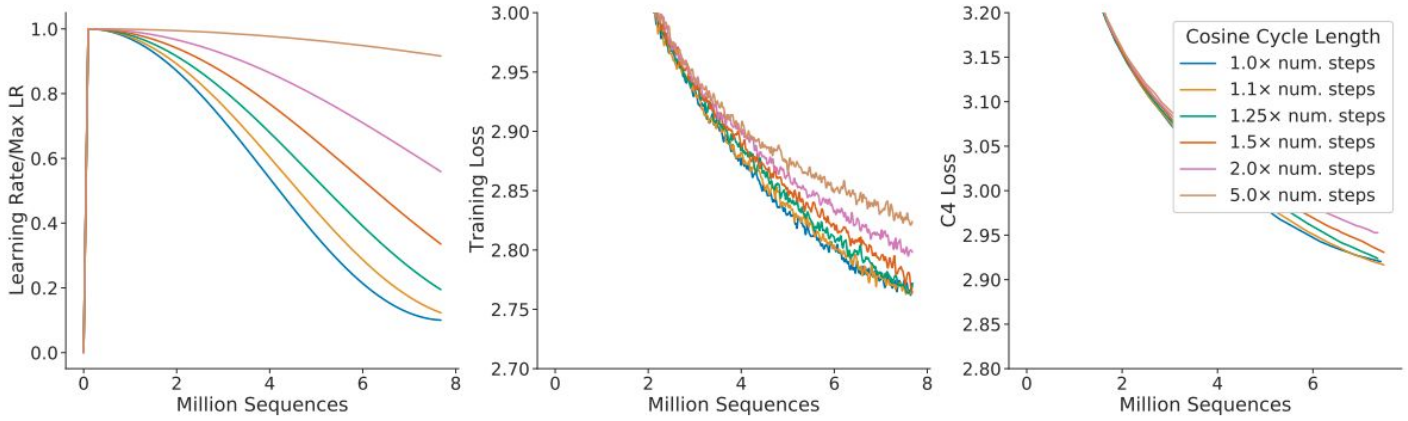
Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27



What causes this discrepancy?

- Kaplan et al didn't take into account **learning rate schedules**
  - Their models were all trained for a fixed number of steps (with a fixed learning rate schedule)
  - Hence their intermediate losses were **overestimates**

- Also their models were significantly smaller compared to Chinchilla



Approach 2: IsoFLOP curves

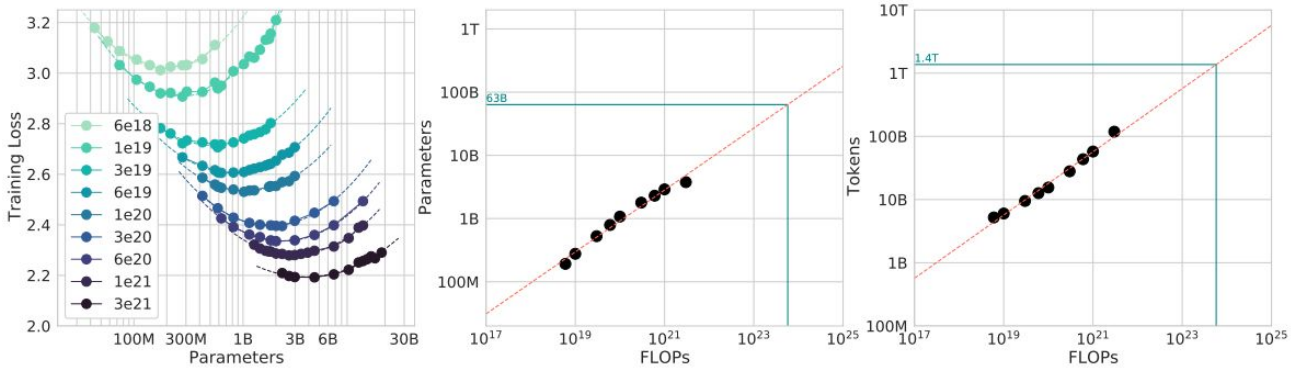


Figure 3 | **IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

Final Result: Chinchilla

A smaller model trained for much longer

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Random	25.0%
Average human rater	34.5%
GPT-3 5-shot	43.9%
Gopher 5-shot	60.0%
***Chinchilla* 5-shot**	**67.6%**
Average human expert performance	89.8%
June 2022 Forecast	57.1%
June 2023 Forecast	63.4%

Table 6 | **Massive Multitask Language Understanding (MMLU).** We report the average 5-shot accuracy over 57 tasks with model and human accuracy comparisons taken from Hendrycks et al. (2020). We also include the average prediction for state of the art accuracy in June 2022/2023 made by 73 competitive human forecasters in Steinhardt (2021).

Note: train-optimal may not be what we want

- Previous scaling laws tell us how to optimally allocate training compute
- But most of the compute is actually spent during inference (after the model is deployed

Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws;
Sardana et al, Neurips 2023

Note: train-optimal may not be what we want

- Previous scaling laws tell us how to optimally allocate training compute
- But most of the compute is actually spent during inference (after the model is deployed
- So it might be better to over-train (a smaller model)

  - **GPT3** – 2 tokens / param
  - **Chinchilla** – 20 tokens / param
  - **LLaMA65B** – 22 tokens / param
  - **Llama 2 70B** – 29 tokens / param
  - **Mistral 7B** – 110 tokens / param
  - **Llama 3 70B** – 215 tokens / param

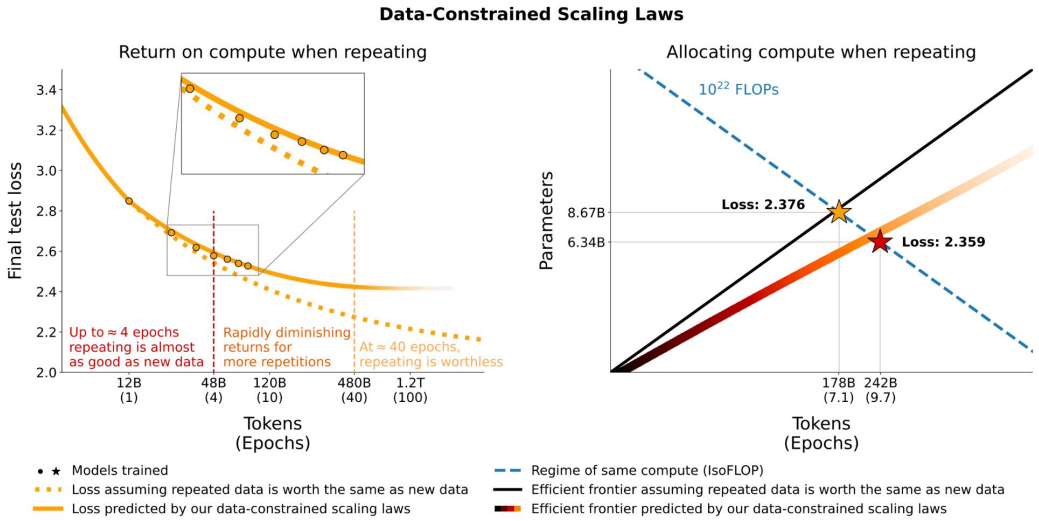
Production LLMs generally train for much longer than suggested by Chinchilla laws

Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws; Sardana et al, Neurips 2023

Data repetition changes scaling laws

Repeated data is worth less than unique data

If you have a fixed amount of data, scale up tokens (repetitions) more than parameters



Scaling Data-Constrained Language Models; Muennighoff et al, Neurips 2023

Recap so far

- Linear relationship between log loss and log data / compute / model-size
- Training lots of small models can help predict optimal hyperparameters for larger model training
- In practice, model providers almost always overtrain
  - Better for inference compute
  - Better with repeated data

How to scale up

- Dense models (regular Transformers)
  - Different types of parallelism
- Sparse models (Mixture-of-Experts)

Types of Parallelism

Scaling Transformers requires us to split their weights across many (often 1000s) of GPUs

Key consideration: computation is cheaper than communication

- So we want to divide our forward and backward passes in a way that minimum synchronization is needed across devices

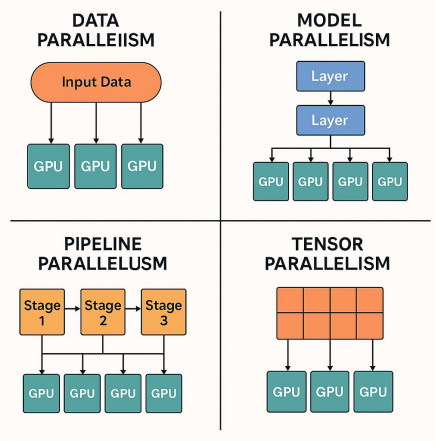


Figure from: link

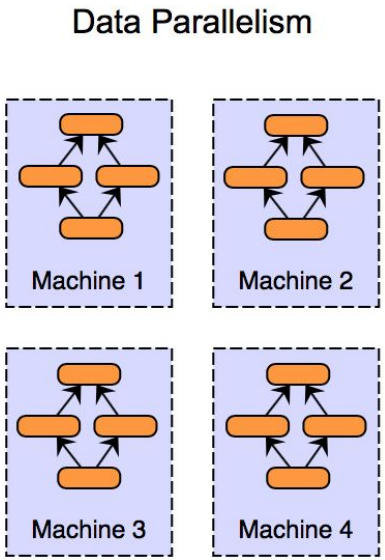
Data Parallelism

Most naive way of parallelizing – copy the model to each device and send different “mini-batches” of data to each

After each forward and backward pass, workers synchronize gradients and model weights

Pros: simple, minimal communication

Cons: Model needs to fit within a single GPU’s memory

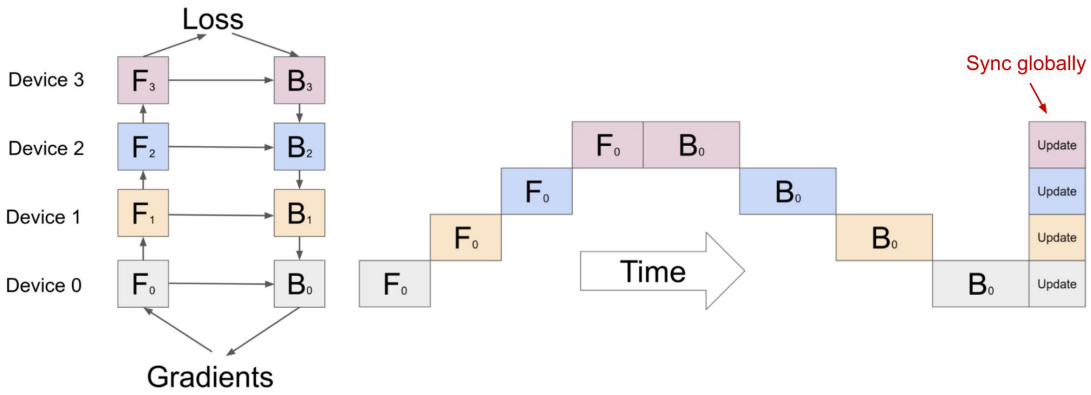


Model Parallelism

Naive version: Send different model layers to different devices

Each device performs the forward and backward computation for a subset of consecutive layers

Cons: Big bubbles of waiting time when GPUs sit idle

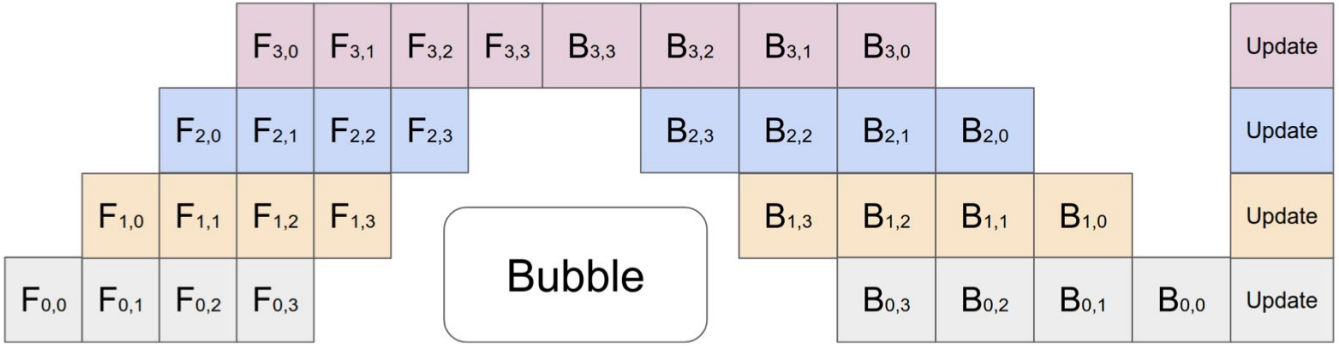


Pipeline Parallelism

Introduced in GPipe paper (2018): Combine model and data parallelism to prevent idle time

Divide the mini-batch into micro-batches and process them sequentially

Pros: If number of micro-batches is more than 4x the number of partitions, the bubble time is negligible

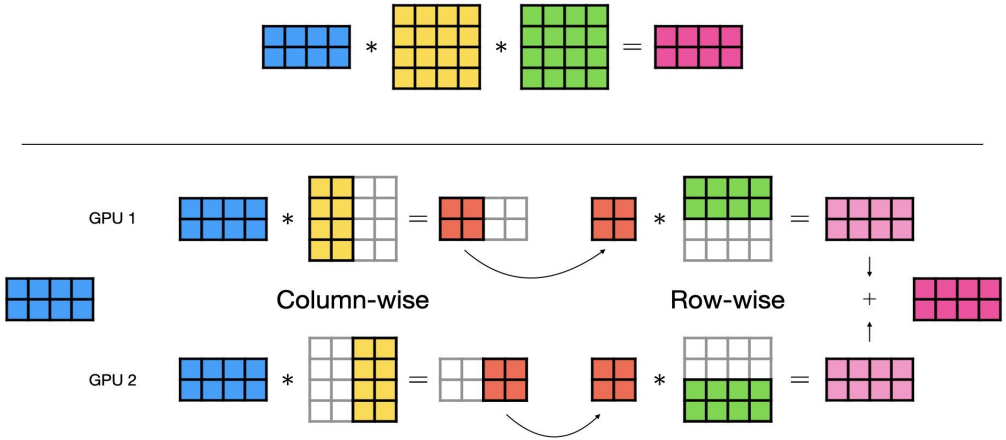


Tensor Parallelism

Introduced in Megatron-LM paper (2020).

Instead of splitting layers across workers, split weight matrices

In Transformers, a lot of the computation is done in the 2-layer FFNs. This can be efficiently parallelized:



Recap

Typical training workloads may utilize any combination of data, tensor and pipeline.

Beyond the network architecture, the topology of the compute cluster also matters:

- Communication between GPUs on a single node (typically 8) is faster, so better for tensor parallelism
- GPUs across nodes have slowed communication, so better for data parallelism

Can we scale model size without scaling compute?

Conditional computation: Activate different parts of a neural network based on the input

- The idea has existed for quite a while (see Jacobs et al, 1991)

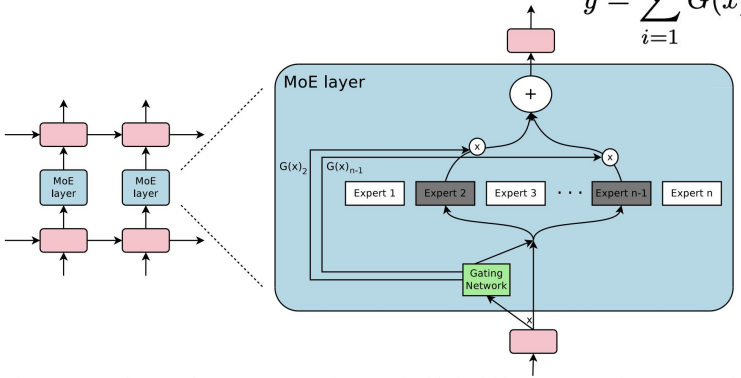
Can we scale model size without scaling compute?

Conditional computation: Activate different parts of a neural network based on the input

- The idea has existed for quite a while (see Jacobs et al, 1991)

- Shazeer et al, 2017 introduced the Mixture-of-Experts (MoE) layer – first application to scale NNs 1000x

$$y = \sum_{i=1}^{n} G(x)_i E_i(x)$$

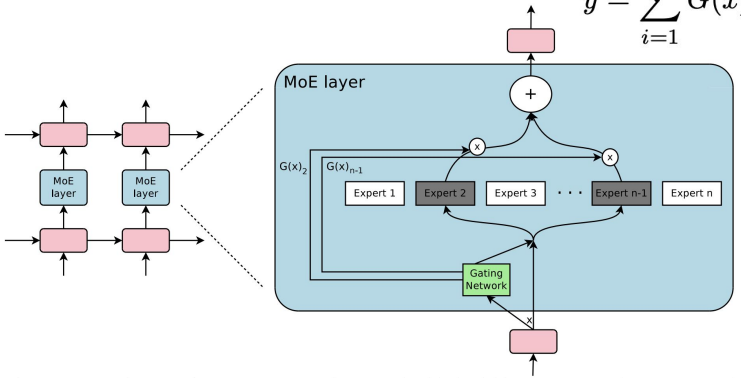


Can we scale model size without scaling compute?

Conditional computation: Activate different parts of a neural network based on the input

- The idea has existed for quite a while (see Jacobs et al, 1991)
- Shazeer et al, 2017 introduced the Mixture-of-Experts (MoE) layer – first application to scale NNs 1000x

$$y = \sum_{i=1}^{n} G(x)_i E_i(x)$$



Each expert is a FFN

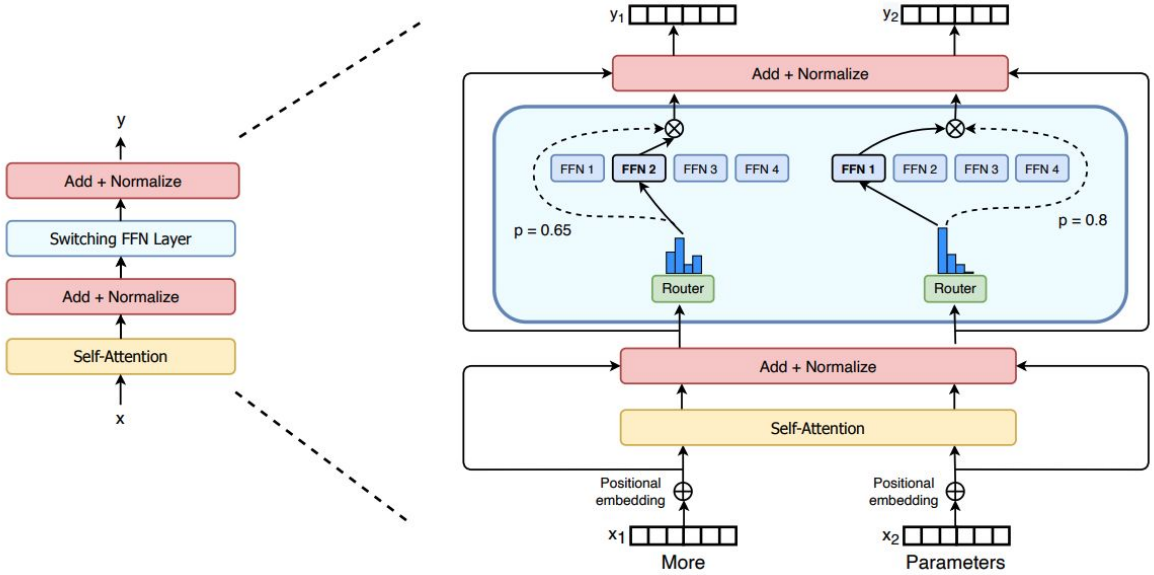
The gating network selects the top k experts to **route** the input through

$$G_\sigma(x) = Softmax(x \cdot W_g)$$

MoE layer applied between two LSTM layers (this was 2017)

MoE Transformer

Replace the FFN in the transformer layer with a MoE



$$h(x) = W_r \cdot x$$

$$p_i(x) = \frac{e^{h(x)_i}}{\sum_j^N e^{h(x)_j}}.$$

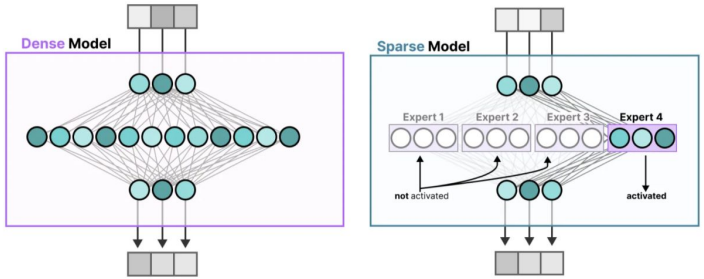
$$y = \sum_{i \in \mathcal{T}} p_i(x) E_i(x).$$

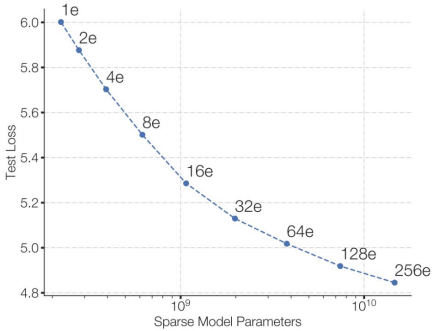
Switch Transformer, Fedus et al, 2022

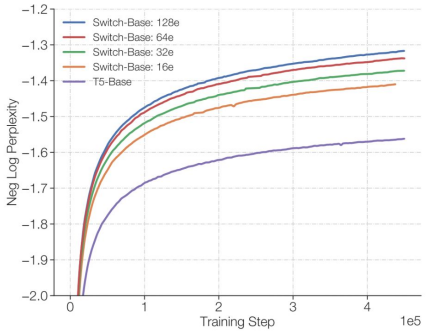
Most LLMs today are MoEs

Why?

- We can add more parameters without increasing FLOPs / example
  - And this improves test loss





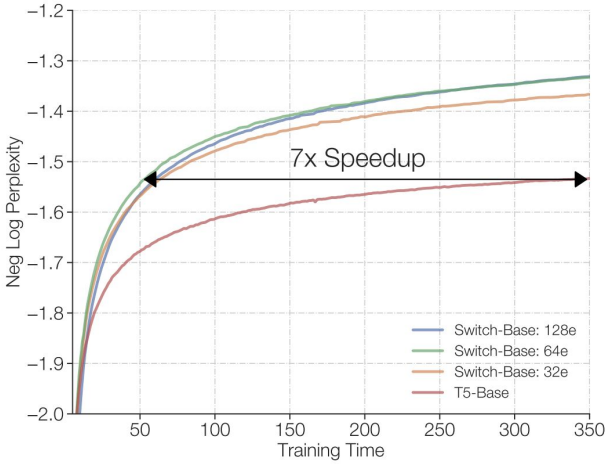


Switch Transformer, Fedus et al, 2022

Most LLMs today are MoEs

Why?

- We can add more parameters without increasing FLOPs / example
- MoEs are faster to train

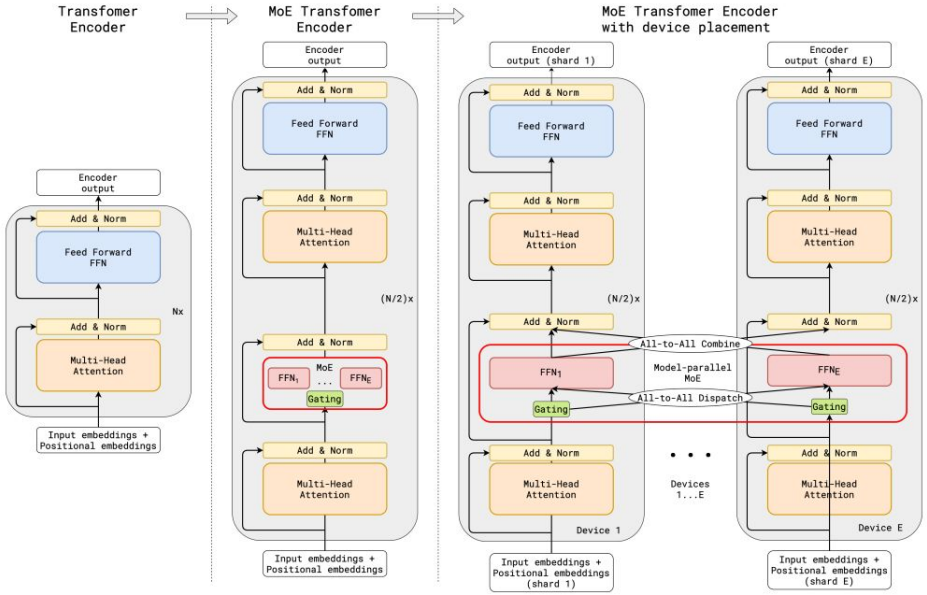


Switch Transformer, Fedus et al, 2022

Most LLMs today are MoEs

Why?

- We can add more parameters without increasing FLOPs / example
- MoEs are faster to train
- Expert parallelization



GShard (2021)

Problem with training MoEs

- **Load balancing**: without special auxiliary losses, only a few experts are selected most of the time
  - This leads to poor utilization of GPUs

Problem with training MoEs

- **Load balancing**: without special auxiliary losses, only a few experts are selected most of the time
  - This leads to poor utilization of GPUs

- **Solution:** Auxiliary losses which encourage balanced utilization of experts
  - **Importance loss:** (Shazeer et al, 2017) reduces the variance in the sum of the gate values across experts

$$Importance(X) = \sum_{x \in X} G(x)$$

Gate values

$$L_{importance}(X) = w_{importance} \cdot CV(Importance(X))^2$$

Problem with training MoEs

- **Load balancing**: without special auxiliary losses, only a few experts are selected most of the time
  - This leads to poor utilization of GPUs

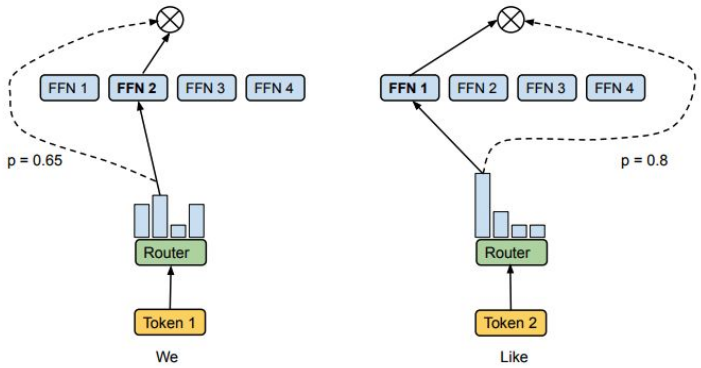
- **Solution:** Auxiliary losses which encourage balanced utilization of experts
  - **Importance loss:** (Shazeer et al, 2017) reduces the variance in the sum of the gate values across experts
  - **Load loss:** (Fedus et al, 2022) encourages uniform routing

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i \qquad f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax}\, p(x) = i\} \qquad P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x).$$

Gate values

Routing function

- Token-choice routing: each token selects its top-k experts

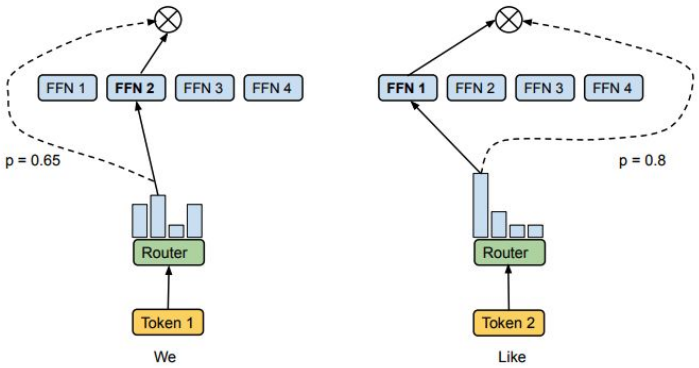


Token-choice

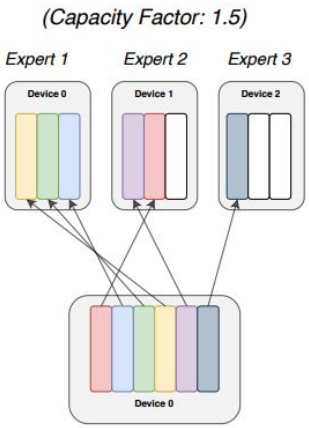
Routing function

- Token-choice routing: each token selects its top-k experts

- Problems:
  - Need to tune the aux loss
  - Still doesn't guarantee balancing, so a capacity factor is needed
  - Dropped tokens and wasted compute
  - Each token has the same number of experts

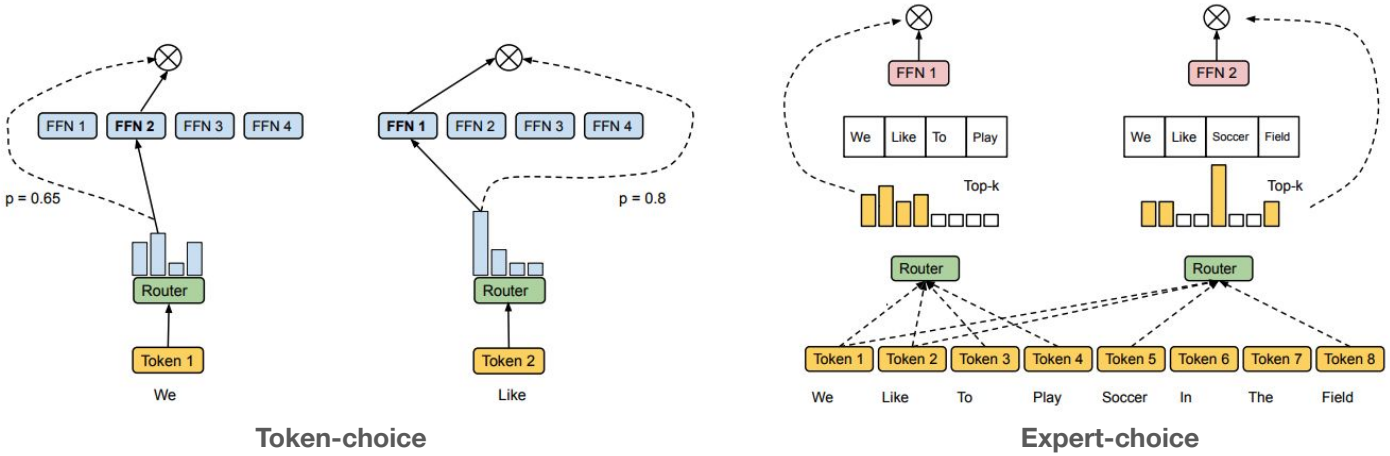


Token-choice



Routing function

- Expert-choice routing: each expert selects k tokens
  - Guarantees load balancing – no auxiliary loss needed
  - Allows variable number of experts per token
- But not good for generation – why?



Summary

- MoEs are sparse neural networks – each input activate a subset of parameters
- Due to this they are more efficient at inference – parameters can be increased without increasing FLOPs
- Routing functions are hard to optimize – but auxiliary loss functions help

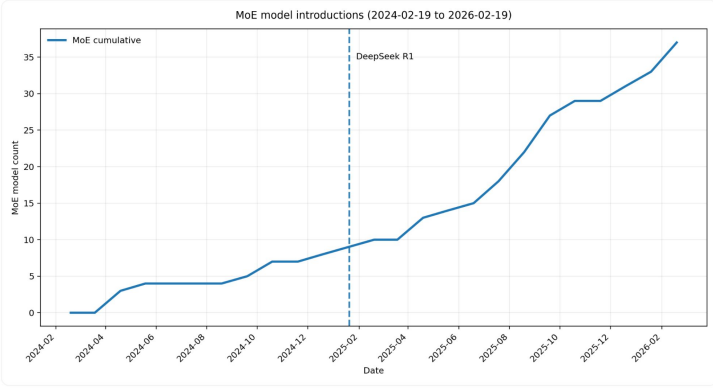


Figure 3: 2-year timeline of MoE model addition to the `transformers` library. DeepSeek R1 marks a clear inflection point.

https://huggingface.co/blog/moe-transformers